



PunkBackup — iPhone Setup, Manual Build (Alternate Method)

This manual shows the PC app in **English** — go to **"Settings"** → **"English"** if it isn't already switched (instant, no restart).

This is the alternate path. Almost everyone should use the main **iPhone Setup Manual** instead — it installs a ready-made Shortcut file in about a minute. Use this guide only if you'd rather build every action by hand, or want to understand exactly what each step does.

Before you start, have these ready

Same requirement as the main manual: this system supports several devices/people (profiles) on the same PC, and **each device needs its own profile and its own token**.

On your PC: 1. Open the app and click **"Start backup"**. 2. In **"Settings"**, click **"+ Add profile"** and name it after THIS device (e.g. `iPhone de María`), then choose its destination folder when prompted. 3. The app shows you (and auto-copies) that profile's **token** — it only works for this one device.

Note down these values:

Value	Where to find it in the app	Example
Server address (same for every profile)	"Address" field	<code>http://[YOUR_PC_NAME].local:8787</code>
Fallback IP (in case the address above doesn't resolve from the iPhone)	"Fallback IP" field	<code>http://192.168.1.50:8787</code>
This profile's token	"Copy token" button on the profile row you just created	a long string of letters/numbers

Your iPhone and your PC must be on the **same WiFi network**.

Step 1 — Create the main Shortcut ("PunkBackup")

1. Open the **Shortcuts** app.
2. **My Shortcuts** tab → + button → **Add Action** (new shortcut).
3. Tap the name at the top and rename it to: **PunkBackup**.
4. Add the following actions **in this order** (search each by name using the magnifying glass):

1) Store the server address and token as variables

- **Text** action → type the server address, e.g. `http://[YOUR_PC_NAME].local:8787`
- **Set Variable** action → name it `ServerURL` → value: the Text above.
- **Text** action → paste the Token **for this profile/device** you copied from the app.
- **Set Variable** action → name it `Token` → value: the Text above.

These are the only two actions you'll ever need to edit again — for example if you regenerate this profile's token from the app ("Regenerate token" button).

2) Tell the server a backup run is starting

- **Get Contents of URL** action:
- URL: `ServerURL + /run/start` (tap the `ServerURL` variable, then type `/run/start` right after it).
- Method: **POST**
- Headers: add one → Key `X-Backup-Token` → Value: the `Token` variable.
- **Get Dictionary Value** action → Key: `run_id` → (applied to the result of the previous step).
- **Set Variable** action → name it `RunID` → value: the result above.

3) Set up the backward block-sweep

`Find Photos` has no built-in pagination — asked for everything at once, it fails outright on a large library (confirmed: no `Limit` = the Shortcut errors out, even doing nothing but counting results). The fix is to sweep the library **backward in bounded blocks**, newest photos first, each block capped at 50 items, advancing a moving date boundary between blocks — all within one run of the Shortcut.

- **Text** action → type `50` (how many blocks of 50 to sweep per run — see the tuning note at the end of this section).
- **Set Variable** → name it `Repeticiones`.
- **Current Date** action.
- **Adjust Date** action → **Add 1 day** to it (so the very first block excludes nothing — "before tomorrow" covers everything you have today).
- **Set Variable** → name it `Limite` (type: Date — keep it as an actual Date value here, not text; the next step compares it directly against each photo's `Date Taken`).
- **Repeat** action (the plain "Repeat X times" kind, **not** "Repeat with Each" — this is the outer loop) → for its count, insert the `Repeticiones` variable instead of typing a fixed number.

Everything below, through the end of section 4, goes **inside** this outer Repeat:

- **Find Photos** action (may be labeled "Filter Photos" on some iOS versions):
- Add a filter: **Date Taken** → **is before** → insert the `Limite` variable.

- Sort by: **Date Taken**. Order: **Latest First** (this is the opposite of what you'd naturally pick — it's what makes the sweep start from your newest photos and work backward).
- **Limit**: turn it **on**, set to **50**. This is mandatory, not optional — **Find Photos** without a **Limit** fails on a large library.
- **Count** action → **Items** in the **Find Photos** result above.
- **If** action: condition = the **Count** result **is 0** (this fires once the sweep has passed your oldest photo — nothing left to check).
- Inside the "If": **Stop This Shortcut** action.
- Nothing needed in "Otherwise" — if the count isn't 0, execution just continues past the "End If" into section 4 below.

4) Loop through each item in the current block: ask first, only upload if missing

Repeat with Each action over the **Find Photos** result from section 3. Inside this inner "Repeat" block:

Step a.0) Set Variable action, as the very first action in this block:

- Value: tap "Repeat Item" and choose the **Date Taken** attribute (leave it as the raw Date — don't run it through **Format Date** here).
- Name the variable **UltimaFecha**. This captures the current item's exact date+time; by the time the loop finishes, it holds the **OLDEST** item's date in this block (since sorted newest-first) — section 4's closing steps use it to move **Limite** for the next block.

Step a) Format Date action:

- Date: tap "Repeat Item" and choose the **Date Taken** attribute.
- Format: **Custom** → type: yyyy-MM-dd 'T' HH:mm:ss
- Store this as a variable named **TakenAt** (**Set Variable** action).
- **Fragile chip, double-check it after saving**: if you later add or move any action **INSIDE** this same "Repeat" block (e.g. step a.0 above), Shortcuts can silently reconfigure this chip to point at a different attribute (like "Name") instead of "Date Taken" — with no red error, no visible warning. The symptom shows up weeks later: ALL new photos land in the same folder (this month's) instead of their real month. If you suspect this, tap the chip inside "Format Date" and confirm it says **Date Taken**, not Name or anything else.

Step a.2) Text action — build the full filename, **with extension** (important: Shortcuts' "File Name" attribute on its own does **not** include the extension — skip this step and you'll end up with files like IMG_1234 instead of IMG_1234.HEIC):

- In the text field, insert: **Repeat Item** → choose the **File Name** attribute → type a period . (no spaces) → insert **Repeat Item** again → choose the **File Extension** attribute.
- It should read something like: [File Name].[File Extension]
- Store this as a variable named **FileName** (**Set Variable** action).

Step b) Get Contents of URL action — the lightweight check, **WITHOUT** the file:

- URL: `ServerURL + /check`
- Method: **POST**
- Request Body: **Form**
- Form fields:
 - `filename` → value: the **FileName** variable (from step a.2 — NOT the bare "File Name" attribute, which is missing the extension).
 - `taken_at` → value: `TakenAt` variable.
- No need to send the file size — Shortcuts has no reliable way to give a plain byte count (it always formats it as something like "1.2 MB"), so the check only uses filename + date. The exact content check happens later, in `/upload`.
- Headers: `X-Backup-Token` → `Token` variable.
- **Get Dictionary Value** action → `key missing` → applied to that result.

Step c) If action: condition = the value above **has any value** (this is how the server tells you "it's missing, upload it"; when a file is already backed up, the server simply omits the missing field entirely, so this condition is automatically false and the "If" is skipped — you do NOT need "is equal to" or to type `false` anywhere, "has any value" is already the right option):

Get Contents of URL (inside the "If"):

- URL: tap the field and insert, IN THIS ORDER, inside the same text field: the **ServerURL** chip → type `/upload?filename=` → insert the **FileName** variable chip (the same one from step a.2 — NOT the bare "File Name" attribute, which is missing the extension) → type `&taken_at=` → the **TakenAt** chip → type `&run_id=` → the **RunID** chip. Inserting the chips directly into the URL field (instead of building the text separately with "Combine Text") makes Shortcuts URL-encode them automatically.
- Method: **POST**
- **Request Body** → change it to **File** (no longer Form).
- In the value field that appears, open the variable bar and insert **Repeat Item** — **once, and don't tap the chip again afterward**. (Tapping it again opens a "Name/Album/Width..." property menu — if that happens and you end up with anything other than plain "Repeat Item", clear it with "Clear Variable" and re-insert it without touching it a second time.) If this chip ever shows up highlighted in red/"broken" in the editor — it can happen after editing actions earlier in the same loop — delete it and re-insert it fresh the same way; a broken reference here silently uploads an empty (0-byte) file instead of erroring visibly.
- Headers: `X-Backup-Token` → `Token` variable.

Step c.1) Get Dictionary Value — check whether that upload failed (videos imported from WhatsApp/other apps sometimes arrive empty, 0 bytes — see the note below):

- **Get Value for:** type `detail`
- **in:** the result of the `Get Contents of URL` above (it appears as "Contents of URL" in the recent variables list).

Step c.2) If (new, nested inside step c's "If"): condition = the result of step c.1 **has any value** (if the direct upload succeeded, the server doesn't send a `detail` field, so this condition is false and the whole block is skipped — same "has any value" idiom as step c itself).

Step c.3) Encode Media (inside this new "If"):

- Item: **Repeat Item** — insert it directly, once, and don't tap it again afterward (same care as always with this chip).
- **Size**: Passthrough (doesn't reduce quality or resolution — it just forces Shortcuts to read the full file, which is exactly what fails for these videos).

Step c.4) Get Contents of URL (second attempt — safest to copy the one above and paste it here, instead of retyping the URL by hand):

- Same URL, same Method POST, same Headers as the original `Get Contents of URL` above.
- **Request Body** → **File** → value: the result of **Encode Media** (step c.3) — NOT Repeat Item this time.

Nothing needed in this nested "If"'s "Otherwise".

(Nothing needed in the outer "Otherwise" branch of step c — if it was already backed up, just move on to the next item.)

Why this retry exists: confirmed on a real device that a video imported from WhatsApp/Messages/other apps sometimes arrives at the server as 0 bytes on the first attempt — Shortcuts fails to read its real data straight from the Photos library. "Encode Media" with **Size: Passthrough** does force that full read to succeed (confirmed by comparing the original video against the processed one with `ffprobe`: same codec, same resolution, same bitrate — no real quality loss). The retry only kicks in when the direct upload fails — camera-native videos (`IMG_XXXX`) almost always upload fine on the first try, without ever touching Encode Media.

Still inside the **inner** "Repeat with Each", after its own "End Repeat" but **before** the outer "End Repeat" from section 3 — these two actions advance the sweep to the next block:

- **Adjust Date** action → **Subtract 1 minute** from **UltimaFecha** (a small safety buffer, so a photo sharing the exact same timestamp as the block boundary — e.g. burst-mode shots — never gets silently skipped; worst case it's re-checked redundantly, which is harmless).
- **Set Variable** action → variable: **Limite** (pick the *existing* `Limite` from the list, don't create a new one with the same name) → value: the result of the `Adjust Date` above.

Tuning Repeticiones: confirmed working with `Repeticiones` up to 50 (~2,500 photos checked in one run) in real testing; `Repeticiones = 180` (~9,000 photos) has also been confirmed not to crash or fail the Shortcut itself. Don't assume a block that's already fully backed up is fast just because it transfers no file — measured directly from real activity logs, an "already backed up" check can still take a real

double-digit number of seconds per item, not a near-instant round trip. See the Troubleshooting Manual's "How fast is a backup, really?" section for the measured numbers (including a per-Repeticiones time table) and why a run that's mostly re-checking already-backed-up content can take longer than one uploading the same number of brand-new items.

Important, non-obvious limitation: Limite always resets to "tomorrow" at the start of every run — there is no memory of where a *previous* run stopped. If a run ends because it used up all its Repeticiones (rather than because it found an empty block and reached your oldest photo), **running it again with the same Repeticiones makes no further progress** — it re-sweeps the exact same newest photos and stops at the exact same point every time. To actually reach older photos, you must **increase Repeticiones**, not just re-run the Shortcut. For a one-time full backfill of a large library, set Repeticiones high enough to cover your whole library in a single run (roughly your total photo count ÷ 50, rounded up — e.g. ~180 for a 9,000-photo library) rather than a small number you plan to re-run — whether iOS can sustain that many blocks in one run hasn't been confirmed for very large values, so increase it gradually and watch the PC app's activity log to see it's keeping up.

5) Close out the run and show you the result

- **Get Contents of URL** action:
- URL: `ServerURL + /run/finish`
- Method: **POST**, Form with field `run_id` = RunID variable.
- Header X-Backup-Token → Token variable.
- **Get Dictionary Value** actions (one per field: `files_new`, `files_skipped`, `files_conflict`) applied to that result.
- **Text** action to build a message, e.g.: `Backup complete \nNew: [files_new]\nAlready had: [files_skipped]\nConflicts: [files_conflict]`
- **Show Notification** (or **Show Result**) action with that text.

Save the shortcut — you can now tap it manually to test it. (See the "Tuning Repeticiones" note in section 3 for how long a run takes on a large library, and how to size the sweep for your first backfill.)

Step 2 — Short "check status" shortcut

Optional but recommended — lets you check status without running a full backup.

1. Create a new shortcut named `Backup Status`.
 2. Repeat the two "Text" + "Set Variable" actions from Step 1.1 (`ServerURL`, `Token`).
 3. **Get Contents of URL** action: URL = `ServerURL + /status`, Method **GET**, header X-Backup-Token → Token.
 4. **Get Dictionary Value** actions for `last_backup_at` and `total_files_backed_up`.
 5. **Show Notification** action with those values.
-

Step 3 — Automate it

Same as the main manual's automation step — once your hand-built PunkBackup shortcut works, go to the **iPhone Setup Manual**'s "Automate it: run on its own when you get home" section and follow it exactly the same way, regardless of how the Shortcut was built.

Notes

- The system **never deletes or modifies anything on your iPhone** — it only reads photos and uploads them.
- The truth about "what's already backed up" lives **only in the PC's destination folder**, never on the iPhone. If you switch your profile's folder/USB drive on the PC, the shortcut automatically re-sends whatever is missing from the new folder — you don't need to do anything different on the iPhone.
- Exact action names can vary slightly between iOS versions; if you can't find an action with the exact name above, search by a keyword instead (e.g. "dictionary", "URL", "repeat").